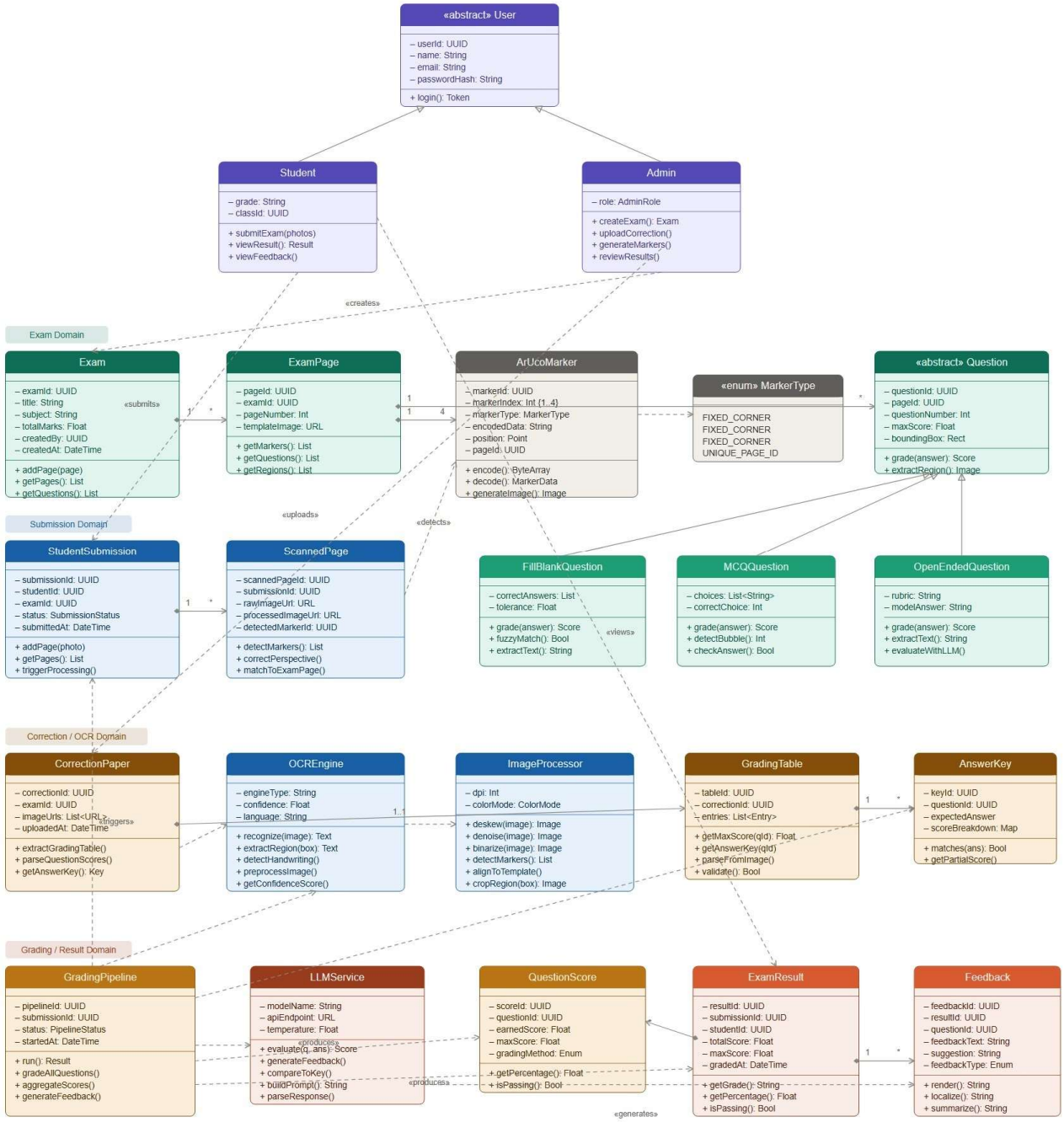
«abstract» User
– userId: UUID
– name: String
– email: String
– passwordHash: String
+ login(): Token
Student
– grade: String
– classId: UUID
+ submitExam(photos)
+ viewResult(): Result
+ viewFeedback()
Admin
– role: AdminRole
+ createExam(): Exam
+ uploadCorrection()
+ generateMarkers()
+ reviewResults()
«creates»
Exam Domain
Exam
– examId: UUID
– title: String
– subject: String
– totalMarks: Float
– createdBy: UUID
– createdAt: DateTime
+ addPage(page)
+ getPages(): List
+ getQuestions(): List
«submits»
ExamPage
– pageId: UUID
– examId: UUID
– pageNumber: Int
– templateImage: URL
+ getMarkers(): List
+ getQuestions(): List
+ getRegions(): List
ArUcoMarker
– markerId: UUID
– markerIndex: Int {1..4}
– markerType: MarkerType
– encodedData: String
– position: Point
– pageId: UUID
+ encode(): ByteArray
+ decode(): MarkerData
+ generateImage(): Image
«enum» MarkerType
FIXED_CORNER
FIXED_CORNER
FIXED_CORNER
UNIQUE_PAGE_ID
«abstract» Question
– questionId: UUID
– pageId: UUID
– questionNumber: Int
– maxScore: Float
– boundingBox: Rect
+ grade(answer): Score
+ extractRegion(): Image
Submission Domain
«uploads»
«detects»
StudentSubmission
– submissionId: UUID
– studentId: UUID
– examId: UUID
– status: SubmissionStatus
– submittedAt: DateTime
+ addPage(photo)
+ getPages(): List
+ triggerProcessing()
ScannedPage
– scannedPageId: UUID
– submissionId: UUID
– rawImageUrl: URL
– processedImageUrl: URL
– detectedMarkerId: UUID
+ detectMarkers(): List
+ correctPerspective()
+ matchToExamPage()
FillBlankQuestion
– correctAnswers: List
– tolerance: Float
+ grade(answer): Score
+ fuzzyMatch(): Bool
+ extractText(): String
«views»
MCQQuestion
– choices: List<String>
– correctChoice: Int
+ grade(answer): Score
+ detectBubble(): Int
+ checkAnswer(): Bool
OpenEndedQuestion
– rubric: String
– modelAnswer: String
+ grade(answer): Score
+ extractText(): String
+ evaluateWithLLM()
Correction / OCR Domain
CorrectionPaper
– correctionId: UUID
– examId: UUID
– imageUrls: List<URL>
– uploadedAt: DateTime
+ extractGradingTable()
+ parseQuestionScores()
+ getAnswerKey(): Key
«triggers»
OCREngine
– engineType: String
– confidence: Float
– language: String
+ recognize(image): Text
+ extractRegion(box): Text
+ detectHandwriting()
+ preprocessImage()
+ getConfidenceScore()
ImageProcessor
– dpi: Int
– colorMode: ColorMode
+ deskew(image): Image
+ denoise(image): Image
+ binarize(image): Image
+ detectMarkers(): List
+ alignToTemplate()
+ cropRegion(box): Image
GradingTable
– tableId: UUID
– correctionId: UUID
– entries: List<Entry>
+ getMaxScore(qId): Float
+ getAnswerKey(qId)
+ parseFromImage()
+ validate(): Bool
AnswerKey
– keyId: UUID
– questionId: UUID
– expectedAnswer
– scoreBreakdown: Map
+ matches(ans): Bool
+ getPartialScore()
Grading / Result Domain
GradingPipeline
– pipelineId: UUID
– submissionId: UUID
– status: PipelineStatus
– startedAt: DateTime
+ run(): Result
+ gradeAllQuestions()
+ aggregateScores()
+ generateFeedback()
LLMService
– modelName: String
– apiEndpoint: URL
– temperature: Float
+ evaluate(q, ans): Score
+ generateFeedback()
+ compareToKey()
+ buildPrompt(): String
+ parseResponse()
«produces»
QuestionScore
– scoreId: UUID
– questionId: UUID
– earnedScore: Float
– maxScore: Float
– gradingMethod: Enum
+ getPercentage(): Float
+ isPassing(): Bool
ExamResult
– resultId: UUID
– submissionId: UUID
– studentId: UUID
– totalScore: Float
– maxScore: Float
– gradedAt: DateTime
+ getGrade(): String
+ getPercentage(): Float
+ isPassing(): Bool
Feedback
– feedbackId: UUID
– resultId: UUID
– questionId: UUID
– feedbackText: String
– suggestion: String
– feedbackType: Enum
+ render(): String
+ localize(): String
+ summarize(): String
«generates»
Inheritance (is-a)
Association / Composition
Dependency «uses / creates»
User Domain
Exam Domain
ArUco Domain
Submission / OCR Domain
Correction Domain
Grading / Result Domain